

wxsum: A command for processing temperature and precipitation data

Jeffrey D. Michler	Anna Josephson	Oscar Barriga Cabanillas
University of Arizona	University of Arizona	World Bank
Tucson, AZ	Tucson, AZ	Washington, D.C.
jdmichler@arizona.edu	jdmichler@arizona.edu	obarriga@worldbank.org
Aleksandr Michuda	Jeffrey C. Oliver	
Swarthmore College	University of Arizona	
Swarthmore, PA	Tucson, AZ	
amichud1@swarthmore.edu	jdmichler@arizona.edu	

Abstract. In this article, we introduce the community-contributed command `wxsum` which processes daily temperature and precipitation data and outputs useful summary statistics as well as measures of weather “shocks.” The command calculates 18 different measures of precipitation and 13 different measures of temperature. These include simple summary statistics, like the mean and standard deviation, as well as measures of shocks, such a z-score, dry spells, and killing degree days (KDD). The goal of the command is to reduce the cost of using weather data in socioeconomic analysis by allowing for quick and accurate calculations of the most common weather metrics used in the literature.

Keywords: st0001, `wxsum`, Earth observation, weather data, summary statistics

1 Introduction

The expanding availability and the decreasing costs of use have made weather data, particularly remotely sensed data, a common input into socioeconomic analysis (Dell et al. 2014; Donaldson and Storeygard 2016). Economists use these ready-made Earth observation products to predict a variety of economics outcomes, such as agricultural output, industrial output, labor productivity, energy demand, health, conflict, and economic growth. But, there are many ways to quantify weather and very little theoretical or systematic guidance on which weather metric is best suited to a given research question and geographic context. The results is that seemingly similar measures of “rainfall,” “temperature,” or “shock” can be based on very different summaries of the underlying weather data (Michler et al. 2021). The variety of metrics used and the size of weather data frames can create data handling and cleaning problem, meaning that the probability of making a coding error in calculating any given metric is non-trivial.

In this article, we introduce the community-contributed command `wxsum` which processes daily temperature and precipitation data and outputs useful summary statistics as well as measures of weather “shocks.” The goal of the command is to allow for easy and error free calculation of the most common weather metrics used in the socioeconomic literature. The `wxsum` command takes in daily temperature or precipitation data,

typically in the form of gridded Earth observation data, and calculates 18 different measures of precipitation and 13 different measures of temperature. These include a number of summary statistics for both temperature and precipitation, like the mean, median, standard deviation, and skew. It also calculates phenomenon-specific statistics, such as max seasonal temperature, growing degree days (GDD), and total seasonal rainfall. The command also calculates a number of measures of shocks. These include deviations from long-run averages, z-scores, dry spells, and killing degree days (KDD).

The **wxsum** command allows the user to set a variety of options so as to customize the weather metrics to their specific geography and research context. One can set the start and end day and month of a season, the bounds for GDD, the threshold of KDD, the number of temperature percentile bins, the threshold volume for what is considered a day with rain, and the length in years of what the user considers “long run.”

The command itself is agnostic in terms of which of the many weather metrics that it calculates a researcher chooses to use. However, in creating the **wxsum** command, we are not endorsing a theory-free or ad hoc approach to incorporating measures of weather into socioeconomic analysis. There is a growing body of literature that raises concerns regarding the use (and misuse) of weather data, particularly when weather is used as a source of identifying exogenous variation (Benami et al. 2026). This literature raises concerns about the way gridded weather data is matched with socioeconomic data (Michler et al. 2022), the existence of measurement error in Earth observation weather data Josephson et al. (2026), and the validity of the assumption that the weather is truly exogenous (Sarsons 2015; Mellon 2025). While incorporating weather data into one’s analysis is now low cost, a naive or unexamined use of that data will continue to have a high cost in terms of the accuracy of one’s analysis and results.

2 Data considerations

The **wxsum** command is designed to process daily weather data in wide format. The expected data structure requires that each row represent a geographic unit—such as a weather station, grid cell, or matched household—and that each column contain a single day’s reading of rainfall or temperature. The daily weather variables must follow a strict naming convention: a user-supplied prefix followed by a date suffix in the form **yyyymmdd**. For example, if the prefix is **rf_**, then the variable containing precipitation data for May 15, 1979, would be named **rf_19790515**. The command parses these variable names to identify the date range of the data and to assemble the appropriate daily variables for each season.

This wide format, while memory-intensive, is a natural representation of the gridded data products that are common in the climate science literature. Most remotely sensed weather products—including the Climate Hazards Group InfraRed Precipitation with Station Data (CHIRPS; Funk et al. 2015), the Climate Prediction Center global daily precipitation data (CPC; Chen et al. 2008), the African Rainfall Climatology (ARC2; Novella and Thiaw 2013), the Tropical Applications of Meteorology using SATellite data (TAMSAT; Tarnavsky et al. 2014), and the Modern-Era Retrospective Analysis

for Research and Applications (MERRA-2; Bosilovich et al. 2016)—provide daily or subdaily readings on a regular grid. These are the data products most commonly used in the socioeconomic literature on weather (Dell et al. 2014). The `wxsum` command is agnostic to the source of the data; it can process data from any of these products or from ground-based weather station networks.

One practical consideration is that the data must be loaded into Stata’s memory before the command is called. Because daily weather data can span decades and cover thousands of grid points, the resulting datasets can be very large. Users working with particularly large datasets may need to ensure that Stata’s memory allocation is sufficient. However, the `wxsum` command does not impose any explicit limit on the number of variables or observations; it will process all valid daily variables that match the supplied prefix.

A second consideration concerns units. The command is agnostic about the units in which weather data are measured. Rainfall may be in millimeters, centimeters, or inches, and temperature may be in degrees Celsius or Fahrenheit, provided that all daily variables are measured consistently. However, threshold options must be expressed in the same units as the data: the lower and upper bounds for growing degree days, the killing degree day base, and the rainy-day threshold must all correspond to the units of the daily variables.

A third consideration concerns the treatment of missing values. Some Earth observation products have incomplete spatial or temporal coverage, and data may also be missing due to sensor malfunction or cloud cover (Dinku 2020). The `wxsum` command handles missing values explicitly: missing daily readings are excluded from all calculations. For rainfall data, missing values do not count as either rainy or dry days and are excluded from the dry-spell calculation. For temperature data, missing values are excluded from degree-day accumulations and bin assignments. The number of nonmissing observations is used as the denominator when computing shares, such as the percentage of rainy days or temperature bin shares.

A fourth consideration is the definition of a season. Agricultural and economic applications typically require weather summaries at the season level rather than the calendar year level, because the relevant growing period varies by crop, geography, and hemisphere. The `wxsum` command allows the user to define a custom season by specifying the start and end month and, optionally, the start and end day. Importantly, the command correctly handles seasons that span calendar-year boundaries—such as a November-to-February season in the Southern Hemisphere—by linking daily variables across consecutive calendar years and labeling the output with the year in which the season begins.

3 The wxsum command

3.1 Syntax

```
wxsum prefix , ini_month(#) fin_month(#) [ , ini_day(#) fin_day(#)
      temp_data rain_data gdd_lo(#) gdd_hi(#) kdd_base(#) bins(#)
      lr_years(#) keep(varlist) save(string) rain_threshold(#) ]
```

prefix is a string identifying the common prefix of the daily weather variables in the dataset.

3.2 Required variables

The dataset in memory must contain daily weather variables named as *prefix*yyyymmdd, where *prefix* is the string supplied to the command and yyyymmdd is an eight-digit date. All such variables must be numeric. The command scans all variables matching the prefix for valid eight-digit date suffixes, determines the date range, and uses only those variables that fall within fully covered seasons.

The user need not drop extraneous variables before calling **wxsum**; the command identifies and processes only the variables whose names match the required pattern. Other variables may remain in the dataset and, if desired, can be retained in the output by specifying the **keep()** option.

3.3 Options

Main. ini_month(#) specifies the starting month of the season as an integer from 1 to 12. This option is required.

fin_month(#) specifies the ending month of the season as an integer from 1 to 12. This option is required. If fin_month is numerically less than ini_month, the command treats the season as spanning across a calendar-year boundary (for example, November to February).

Options. ini_day(#) specifies the starting day of the season. The default is 1.

fin_day(#) specifies the ending day of the season. The default is 1.

temp_data specifies that the data contains temperature readings and that the command should generate temperature-specific output variables (see *Output* below). This option is mutually exclusive with rain_data; exactly one of the two must be specified.

rain_data specifies that the data contains precipitation readings and that the command should generate rainfall-specific output variables. This option is mutually

exclusive with `temp_data`.

`gdd_lo(#)` sets the lower temperature threshold for the growing degree day (GDD) calculation. This option is required when `temp_data` is specified. GDD for each day is calculated as $\min(\max(T - gdd_lo, 0), gdd_hi - gdd_lo)$, where T is the daily temperature.

`gdd_hi(#)` sets the upper temperature threshold for the GDD calculation. This option is required when `temp_data` is specified, and must be strictly greater than `gdd_lo()`.

`kdd_base(#)` sets the temperature threshold for calculating killing degree days (KDD). KDD for each day is $\max(T - kdd_base, 0)$, summed over the season. The default is 0, meaning KDD are not calculated unless a positive threshold is specified.

`bins(#)` sets the number of equal-sized percentile bins into which the daily temperature distribution is divided for each season. Allowed values are integers from 4 to 10. The default is 4.

`lr_years(#)` sets the number of strictly preceding years used as the rolling window for computing long-run average deviations and z-scores. The default is 10; the maximum is 50. If the available data do not contain enough preceding seasons to fill the window, deviations and z-scores are not computed for those early years.

`keep(varlist)` specifies variables from the original dataset to retain in the output alongside the generated weather variables. This is useful for preserving identifiers such as household IDs, grid-cell coordinates, or administrative-unit codes.

`save(filename)` saves the resulting dataset to disk at the specified path, replacing any existing file with the same name.

`rain_threshold(#)` sets the volume threshold (in the units of the data) above which a day is classified as a rainy day. The default is 1. Missing rainfall values are excluded from rain-day, no-rain-day, percentage, and dry-spell calculations.

3.4 Output

The `wxsum` command generates season-level summary variables that are added to the dataset in memory. Each variable is suffixed with the four-digit year in which the season begins (for example, `mean_1995`, `total_2003`). This section describes the output variables separately for rainfall and temperature data.

Rainfall variables

When `rain_data` is specified, the command generates the following 18 variables for each complete season:

Variable	Description
<code>mean_YYYY</code>	Mean daily rainfall in the season
<code>median_YYYY</code>	Median daily rainfall in the season
<code>sd_YYYY</code>	Standard deviation of daily rainfall in the season
<code>skew_YYYY</code>	Skewness of daily rainfall ^a
<code>mean_mo_total_YYYY</code>	Mean of monthly rainfall totals in the season
<code>median_mo_total_YYYY</code>	Median of monthly rainfall totals in the season
<code>sd_mo_total_YYYY</code>	Standard deviation of monthly rainfall totals
<code>skew_mo_total_YYYY</code>	Skewness of monthly rainfall totals ^a
<code>total_YYYY</code>	Total seasonal rainfall
<code>dev_total_YYYY</code>	Deviation from long-run average of total seasonal rainfall
<code>z_total_YYYY</code>	Z-score of total seasonal rainfall
<code>raindays_YYYY</code>	Number of rainy days in the season
<code>norain_YYYY</code>	Number of nonmissing days without rain
<code>dev_raindays_YYYY</code>	Deviation from long-run average of rainy days
<code>dev_norain_YYYY</code>	Deviation from long-run average of days without rain
<code>pct_raindays_YYYY</code>	Share of nonmissing days with rain
<code>dev_pct_raindays_YYYY</code>	Deviation from long-run average of rain-day share
<code>dry_YYYY</code>	Length of the longest intra-seasonal dry spell (consecutive days below threshold)

^a Skewness is computed as $(\bar{x} - \tilde{x})/s$, a nonparametric approximation of distributional asymmetry (Pearson's second coefficient of skewness without the factor of 3).

The deviation and z-score variables (`dev_` and `z_` prefixes) are computed with a rolling window of `lr_years` strictly preceding seasons. Let w_{it} denote a seasonal weather statistic for location i in the season beginning in year t , and let L denote the number of prior years specified in `lr_years()`. The rolling long-run mean and standard deviation are

$$\bar{w}_{it}^L = L^{-1} \sum_{l=1}^L w_{i,t-l} \quad \text{and} \quad s_{it}^L = \left\{ (L-1)^{-1} \sum_{l=1}^L (w_{i,t-l} - \bar{w}_{it}^L)^2 \right\}^{1/2}$$

The deviation is $w_{it} - \bar{w}_{it}^L$, and the z-score is $(w_{it} - \bar{w}_{it}^L)/s_{it}^L$. The use of strictly prior years avoids including the current realization in its own benchmark. If fewer than L preceding seasons are available, deviations and z-scores are not generated for those early seasons.

Temperature variables

When `temp_data` is specified, the command generates the following variables for each complete season:

Variable	Description
<code>mean_YYYY</code>	Mean daily temperature in the season
<code>median_YYYY</code>	Median daily temperature in the season
<code>sd_YYYY</code>	Standard deviation of daily temperature in the season
<code>skew_YYYY</code>	Skewness of daily temperature ^a
<code>max_YYYY</code>	Maximum daily temperature in the season
<code>gdd_YYYY</code>	Accumulated growing degree days (GDD) ^b
<code>dev_gdd_YYYY</code>	Deviation from long-run average GDD
<code>z_gdd_YYYY</code>	Z-score of GDD
<code>kdd_YYYY</code>	Accumulated killing degree days (KDD) ^c
<code>dev_kdd_YYYY</code>	Deviation from long-run average KDD
<code>z_kdd_YYYY</code>	Z-score of KDD
<code>tempbinBB_YYYY</code>	Share of observed days in temperature bin <i>BB</i> ^d
<code>binmean_BB</code>	Mean bin share across all seasons
<code>binsd_BB</code>	Standard deviation of bin share across all seasons

^a Computed as $(\bar{x} - \tilde{x})/s$.

^b $GDD = \sum_{d=1}^D \min(\max(T_d - gdd_lo, 0), gdd_hi - gdd_lo)$, where D is the number of days in the season.

^c $KDD = \sum_{d=1}^D \max(T_d - kdd_base, 0)$. Only generated when `kdd.base()` is set to a positive value.

^d *BB* is a zero-padded two-digit bin number (01, 02, ...). Bin boundaries are defined by equal-frequency percentiles of the within-season daily temperature distribution.

The temperature bin variables divide the within-season distribution of daily temperatures into `bins()` equal-frequency percentile groups and record the share of observed days falling into each bin. For instance, with the default of four bins, the boundaries are the 25th, 50th, and 75th percentiles of observed daily temperatures in each season. Across all processed seasons, the command also computes the cross-season mean (`binmean_BB`) and standard deviation (`binsd_BB`) of each bin's share. Temperature bins are a flexible, nonparametric method for capturing the nonlinear relationship between temperature and economic outcomes (Deschène and Greenstone 2007).

4 Examples

We illustrate the use of `wxsum` with two examples using the sample datasets bundled with the package: `rain.dta` (daily rainfall) and `temp.dta` (daily temperature). These datasets are installed alongside the command and are available via the GitHub repository.

4.1 Rainfall example

The rainfall dataset contains daily precipitation observations with the prefix `rf_`. We define a season running from the 15th of May to the 15th of October and request that the output be saved to disk:

```
. use rain.dta, clear

. wxsum rf_, ini_month(05) fin_month(10) ini_day(15) fin_day(15) ///
>     rain_data save(rainfall_stats.dta)
Saving data set as rainfall_stats.dta
file rainfall_stats.dta saved
```

The command identifies all complete seasons within the data’s date range and creates 18 rainfall variables per season. Deviation and z-score variables are generated for seasons with at least 10 preceding seasons of data (the default `lr_years`).

4.2 Temperature example

The temperature dataset uses the prefix `tmp_`. Here we define a cross-year season from November to February and specify GDD bounds of 8°C and 32°C. We retain the household identifier `hhid` in the output:

```
. use temp.dta, clear

. wxsum tmp_, ini_month(11) fin_month(02) temp_data ///
>     gdd_lo(8) gdd_hi(32) keep(hhid)
```

Because the season spans November through February, the command links daily variables across calendar-year boundaries. The output is labeled with the year in which each season begins (for example, `gdd_2001` refers to the November 2001–February 2002 season). With the default of 4 temperature bins, the command creates variables `tempbin01_YYYY` through `tempbin04_YYYY` for each season, as well as the summary variables `binmean_01` through `binmean_04` and `binsd_01` through `binsd_04`.

5 Conclusions

We have introduced the `wxsum` command, a community-contributed Stata program for processing daily temperature and precipitation data into season-level summary statistics and shock measures. The command standardizes the construction of summary statistics, threshold-based heat measures, rainy-day counts, dry spells, and rolling weather-shock measures while leaving the substantive choices—data source, spatial match, season definition, units, and thresholds—with the researcher. This division of labor is useful: the command lowers the probability of programming errors in repetitive weather-processing tasks, but it does not substitute for careful research design.

Users can customize the season definition, the thresholds for growing and killing degree days, the number of temperature bins, and the length of the long-run window for computing deviations and z-scores. The command handles cross-year seasons, missing data, and large datasets without requiring users to reshape or preprocess their data beyond the standard wide format.

The `wxsum` command can be installed from its GitHub repository (<https://github.com/jdavidm/wxsum>) using the following Stata command:


```
. net install wxsum, from("https://raw.githubusercontent.com/jdavidm/wxsum/master/") replace
```

Sample datasets (`rain.dta` and `temp.dta`) are included in the repository for testing and illustration. Bug reports and feature requests can be submitted via the repository's issue tracker.

6 References

- Benami, E., J. D. Michler, A. Josephson, M. Cecil, and G. Maskell. 2026. The Uses (and Misuses) of Earth Observation Data on Weather and Vegetation. In *Geospatial Impact Evaluation in Practice*, ed. A. BenYishay and K. Singh. Taylor & Francis.
- Bosilovich, M., R. Lucchesi, and M. Suarez. 2016. MERRA-2: File Specification. GMAO Office Note No. 9 (Version 1.1). http://gmao.gsfc.nasa.gov/pubs/office_notes.
- Chen, M., W. Shi, P. Xie, V. B. Silva, V. E. Kousky, R. W. Higgins, and J. E. Janowiak. 2008. Assessing objective techniques for gauge-based analyses of global daily precipitation. *Journal of Geophysical Research* 113: D04110.
- Dell, M., B. F. Jones, and B. A. Olken. 2014. What Do We Learn from the Weather? The New Climate-Economy Literature. *Journal of Economic Literature* 52(3): 740–798.
- Deschêne, O., and M. Greenstone. 2007. The Economic Impacts of Climate Change: Evidence from Agricultural Output and Random Fluctuations in Weather. *American Economic Review* 97(1): 354–85.
- Dinku, T. 2020. Remote sensing of African rainfall. *Oxford Research Encyclopedia of Climate Science* 30 July.
- Donaldson, D., and A. Storeygard. 2016. The View from Above: Applications of Satellite Data in Economics. *Journal of Economic Perspectives* 30(4): 171–98.
- Funk, C., P. Peterson, M. Landsfeld, D. Pedreros, J. Verdin, S. Shukla, G. Husak, J. Rowland, L. Harrison, A. Hoell, and J. Michaelsen. 2015. The climate hazards infrared precipitation with stations—a new environmental record for monitoring extremes. *Scientific Data* 2: 150066.
- Josephson, A., J. D. Michler, T. Kilic, and S. Murray. 2026. The mismeasure of weather: Using earth observation data for estimation of socioeconomic outcomes. *Journal of Development Economics* 178: 103553.
- Mellon, J. 2025. Rain, Rain, Go Away: 194 Potential Exclusion-Restriction Violations for Studies Using Weather as an Instrumental Variable. *American Journal of Political Science* 69(3): 881–898.
- Michler, J. D., A. Josephson, T. Kilic, and S. Murray. 2021. Estimating the Impact of Weather on Agriculture. World Bank Policy Research Working Paper, No. 9867.

- . 2022. Privacy protection, measurement error, and the integration of remote sensing and socioeconomic survey data. *Journal of Development Economics* 158: 102927.
- Novella, N. S., and W. M. Thiaw. 2013. African Rainfall Climatology Version 2 for Famine Early Warning Systems. *Journal of Applied Meteorology and Climatology* 52(3): 588–606.
- Sarsons, H. 2015. Rainfall and Conflict: A Cautionary Tale. *Journal of Development Economics* 115: 62–72.
- Tarnavsky, E., D. Grimes, R. Maidment, E. Black, R. P. Allan, M. Stringer, R. Chadwick, and F. Kayitakire. 2014. Extension of the TAMSAT Satellite-Based Rainfall Monitoring over Africa and from 1983 to Present. *Journal of Applied Meteorology and Climatology* 53(12): 2805–22.

About the authors

Jeffrey D. Michler is an Associate Professor in the Department of Agricultural and Applied Economics at the University of Arizona.

Anna Josephson is an Associate Professor in the Department of Agricultural and Applied Economics at the University of Arizona.

Aleksandr Michua is a Visiting Assistant Professor at the Department of Economics at Swarthmore College.

Oscar Barriga Cabanillas is an Economist at the World Bank.

Jeffrey C. Oliver is a Data Science Specialist in the University of Arizona Libraries.